



JOBSHEET V

BRUTE FORCE AND DIVIDE CONQUER

5.1 Learning Outcome

After completing this practical session, students will be able to develop algorithms using Brute Force and Divide and Conquer approaches. They will also be able to implement these algorithms effectively in problem-solving scenarios.

5.2 Calculating Factorial Using Brute Force and Divide and Conquer Algorithms

There is a class diagram as follows:

Factorial
factorialBF(): int factorialDC(): int

Based on the class diagram above, a Java program will be created to calculate the factorial of a number using two different algorithms: Brute Force and Divide and Conquer. The calculation process differs between these two approaches, as illustrated below:

- Steps for computing factorial using the **Brute Force** algorithm:

$$20! = 20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1$$

- Steps for computing factorial using the Divide and Conquer method:

$$20! = (20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1)$$

5.2.1. Experiment Steps

- Create a new project named **BruteForceDivideConquer** (or continue the previous project) and a package called **week5**.
- Create a new class, `Factorial`
- Complete the `Factorial` class with the attributes and methods as illustrated in the class diagram above.



- a) Create faktorialBF() method:

```
int faktorialBF(int n){
    int facto = 1;
    for(int i=1; i<=n; i++){
        facto = facto * i;
    }
    return facto;
}
```

- b) Create faktorialDC() method:

```
int faktorialDC(int n){
    if(n==1){
        return 1;
    }else{
        int facto = n * faktorialDC(n-1);
        return facto;
    }
}
```

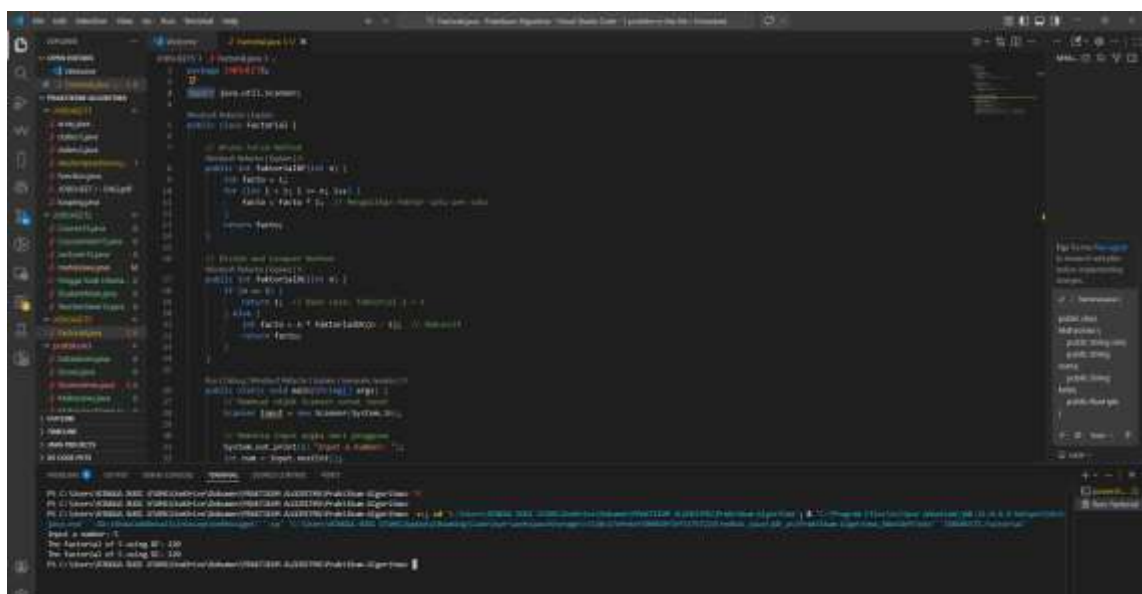
- c) Run the Faktorial class by creating a new class named MainFaktorial and executing it.
d) Add main() function within MainFaktorial class, and get the user input:

```
Scanner input = new Scanner(System.in);
System.out.print(s:"Input a number: ");
int num = input.nextInt();
```

- e) Then, create an object of Faktorial and call the faktorialDC() method and faktorialBF() method:

```
Faktorial fk = new Faktorial();
System.out.println("The faktorial of "+num+" using BF: "+fk.faktorialBF(num));
System.out.println("The faktorial of "+num+" using DC: "+fk.faktorialDC(num));
```

- d) Make sure that no error appears!

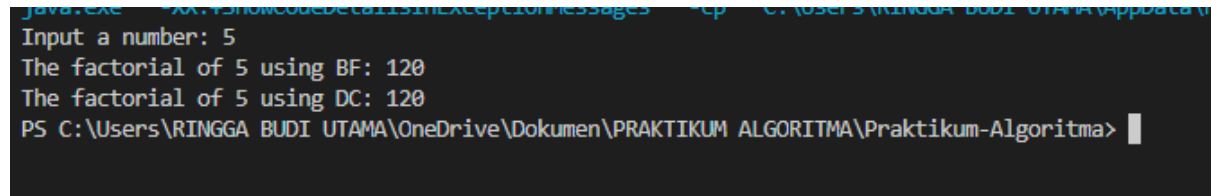




5.2.2. Verification of Experiment Results

Compare the output of your compiled program with the following image.

```
Input a number: 5
The factorial of 5 using BF: 120
The factorial of 5 using DC: 120
```



5.2.3. Questions

1. In the base case of the Divide and Conquer algorithm for calculating factorial, explain the differences in the code structure between the `if` and `else` conditions!
2. Is it possible to modify the loop in the `factorialBF()` method to use an alternative to the `for` loop? Please explain and give example if needed!
3. Please explain the differences between `facto = facto*i;` and `int facto = n * factorialDC(n-1);`!
4. Make a conclusion about the differences in how each method works. `factorialBF()` and `factorialDC()`!

Answer :

1. Perbedaan `if` dan `else` dalam Divide and Conquer

- **if:** Menangani **kasus dasar**, yaitu saat `n == 1`, dan mengembalikan 1.
- **else:** Menangani **rekursi**, mengalikan `n` dengan hasil rekursi untuk `n-1`.

2. Alternatif selain `for` loop di `factorialBF()`

- Anda bisa menggunakan **while loop** untuk iterasi, atau menggunakan **rekursi** untuk mengganti perulangan.

3. Perbedaan `facto = facto * i;` dan `int facto = n * factorialDC(n-1);`

- **facto = facto * i;** Mengalikan secara langsung dalam **iterasi**.
- **int facto = n * factorialDC(n-1);** Mengalikan menggunakan **rekursi**, memecah masalah menjadi sub-masalah.



4. Kesimpulan tentang faktorialBF() dan faktorialDC()

- **faktorialBF()**: Menggunakan **iterasi** untuk menghitung faktorial, lebih sederhana.
- **faktorialDC()**: Menggunakan **rekursi** untuk membagi masalah, lebih elegan tapi lebih kompleks.

5.3 Calculating Exponentiation Using Brute Force and Divide and Conquer Algorithms

In this practical session, we will create a Java program to calculate the exponentiation of a number using two different algorithms: Brute Force and Divide and Conquer. This session will utilize an Array of Objects to manage multiple objects, unlike the previous session, which focused only on a single factorial object.

5.3.1. Experiment Steps

1. In the **week5** package, create a new class named **Power**. Inside this class, define attributes for the base number and its exponent.

```
int baseNumber, exponent;
```

2. Add parameterized constructor

```
Power(int n, int e){
    baseNumber = n;
    exponent = e;
}
```

3. Add **powerBF()** method

```
int powerBF(int n, int e){
    int result = 1;
    for(int i=0; i<e; i++){
        result = result*n;
    }
    return result;
}
```

4. Add **powerDC()** method

```
int powerDC(int n, int e){
    if(e==1){
        return n;
    }else{
        if(e%2==1){
            return (powerDC(n, e/2)*powerDC(n, e/2)*n);
        }else{
            return (powerDC(n, e/2)*powerDC(n, e/2));
        }
    }
}
```

- Next, create a new class that contains the `main` method. This class can be named `PowerMain`. Add code in the main class to input the number of elements for which the exponentiation will be calculated.

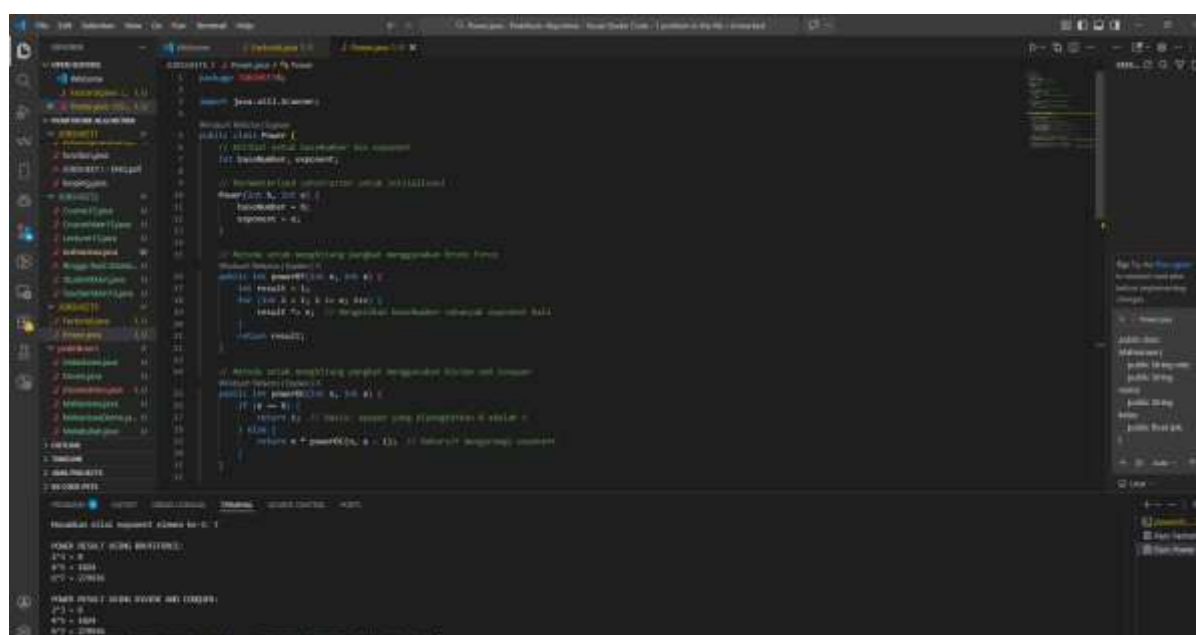
```
Scanner input = new Scanner(System.in);
System.out.print(s:"Input element number: ");
int elemen = input.nextInt();
```

- The value obtained in step 5 will be used to instantiate an array of objects. In the following code, add a process to fill the array with multiple base numbers along with their respective exponents.

```
Power[] png = new Power[elemen];
for(int i=0;i<elemen;i++){
    System.out.print("Input base number for "+(i+1)+"th element: ");
    int basis = input.nextInt();
    System.out.print("Input exponent for "+(i+1)+"th element: ");
    int exp = input.nextInt();
    png[i] = new Power(basis, exp);
}
```

- Call `powerBF()` and `powerDC()` method to perform power calculation using both brute force and DC approach!

```
System.out.println(x:"POWER RESULT USING BRUTEFORCE:");
for (Power p : png) {
    System.out.println(p.baseNumber+"^"+p.exponent+": "+p.powerBF(p.baseNumber, p.exponent));
}
System.out.println(x:"POWER RESULT USING DIVIDE AND CONQUER:");
for (Power p : png) {
    System.out.println(p.baseNumber+"^"+p.exponent+": "+p.powerDC(p.baseNumber, p.exponent));
}
```





5.3.2. Verification of Experiment Results

The result must be like this:

```
Masukkan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
```

5.3.3. Questions

1. Explain the differences between the two methods created `powerBF()` dan `powerDC()` !
2. Does the **combine** stage exist in the provided code? Show the relevant part!
3. In the `powerBF()` method, parameters are used to pass the base number and its exponent, even though the `Power` class already contains attributes for these values (`baseNumber` and `exponent` attribute). Do you think it is still relevant for the method to have parameters? Could the method be implemented without parameters instead? If so, how would the `powerBF()` method be structured without parameters?
4. Summarize how the `powerBF()` and `powerDC()` methods work!

Answer :

1. Perbedaan antara metode `powerBF()` dan `powerDC()`

- **powerBF() (Brute Force):**
 - Metode ini menggunakan pendekatan **iteratif** (loop) untuk menghitung pangkat.
 - Mengalikan **base number** dengan dirinya sendiri sebanyak **exponent** kali.
 - Pendekatan ini langsung dan sederhana.
- **powerDC() (Divide and Conquer):**
 - Metode ini menggunakan pendekatan **rekursif** untuk menghitung pangkat.



- Memecah masalah besar menjadi masalah yang lebih kecil dengan mengurangi **exponent** secara bertahap hingga mencapai 0.
- Pendekatan ini lebih elegan dan efisien dalam hal pembagian masalah.

Perbedaan utama:

- **powerBF()** menggunakan **iterasi** untuk menghitung hasil secara langsung.
- **powerDC()** menggunakan **rekursi**, membagi masalah menjadi sub-masalah yang lebih kecil dan menggabungkan hasilnya.

2. Apakah tahap "combine" ada dalam kode yang diberikan? Tunjukkan bagian relevannya!

Dalam algoritma **Divide and Conquer**, tahap "**combine**" terjadi ketika hasil perhitungan dari sub-masalah digabungkan. Meskipun tidak ada tahap eksplisit yang disebut "combine", tahap ini terjadi secara otomatis saat **rekursi** mengembalikan hasil yang digabungkan.

Bagian relevan:

- Hasil rekursif pada **powerDC()** digabungkan secara otomatis ketika setiap pemanggilan rekursif mengembalikan hasil dan melanjutkan ke tingkat rekursi yang lebih tinggi, hingga akhirnya hasil faktorial atau pangkat dikembalikan.

3. Apakah masih relevan untuk menggunakan parameter di metode powerBF()? Bisakah metode ini diimplementasikan tanpa parameter?

- **Relevansi menggunakan parameter:** Menggunakan parameter di metode **powerBF()** sangat relevan karena memungkinkan fungsi untuk menghitung pangkat dengan nilai yang diberikan secara dinamis, bukan berdasarkan nilai yang sudah ada dalam objek. Ini memberikan fleksibilitas, memungkinkan penggunaan metode ini di berbagai konteks.
- **Implementasi tanpa parameter:** Metode **powerBF()** bisa diubah untuk menggunakan **atribut kelas** yang sudah ada, yaitu **baseNumber** dan **exponent**. Dengan cara ini, metode tidak memerlukan parameter eksternal, karena nilai tersebut sudah ada di dalam objek kelas **Power**.

4. Ringkasan cara kerja metode powerBF() dan powerDC()

- **powerBF() (Brute Force):**
 - Menggunakan **iterasi** untuk menghitung pangkat.
 - Pada setiap langkah, **baseNumber** dikalikan dengan dirinya sendiri sebanyak **exponent** kali.
 - Pendekatan ini mudah dipahami dan langsung menghitung hasilnya.



- **powerDC() (Divide and Conquer):**
 - Menggunakan **rekursi** untuk menghitung pangkat.
 - Membagi masalah menjadi sub-masalah lebih kecil dengan mengurangi **exponent** sampai mencapai 0, di mana proses rekursi berhenti dan hasilnya dikembalikan.
 - Pendekatan ini memanfaatkan pembagian masalah dan sering lebih efisien untuk masalah yang lebih besar.

Kesimpulan:

- **powerBF()** lebih langsung dan sederhana, menggunakan iterasi untuk menghitung pangkat.
- **powerDC()** menggunakan pendekatan rekursif yang lebih elegan dan memecah masalah menjadi bagian yang lebih kecil.

5.4 Calculating Array Sum Using Brute Force and Divide and Conquer Algorithms

In this experiment, we will practice how the divide, conquer, and combine processes are applied in a case study of calculating a company's profit over several months.

5.4.1. Experiment Steps

1. In the **week5** package, create a new class named **Sum**. Add the parametrized constructor as well .

```
double profits[];

Sum(int el){
    profits = new double[el];
}
```

2. Add **totalBF()** method which will calculate the total value of the array using an iterative approach.

```
double totalBF(){
    double total=0;
    for(int i=0;i<profits.length;i++){
        total = total+profits[i];
    }
    return total;
}
```

3. Add **totalDC()** method that will calculate the total value of the array using Divide and Conquer

```
double totalDC(double arr[], int l, int r){
    if(l==r){
        return arr[l];
    }

    int mid = (l+r)/2;
    double lsum = totalDC(arr, l, mid);
    double rsum = totalDC(arr, mid+1, r);
    return lsum+rsum;
}
```

4. Create a new class named `SumMain`. This class should contain the `main` method, where users can specify the number of months for which the profit will be calculated. Additionally, instantiate an object within this class to access attributes and methods from the `Sum` class.

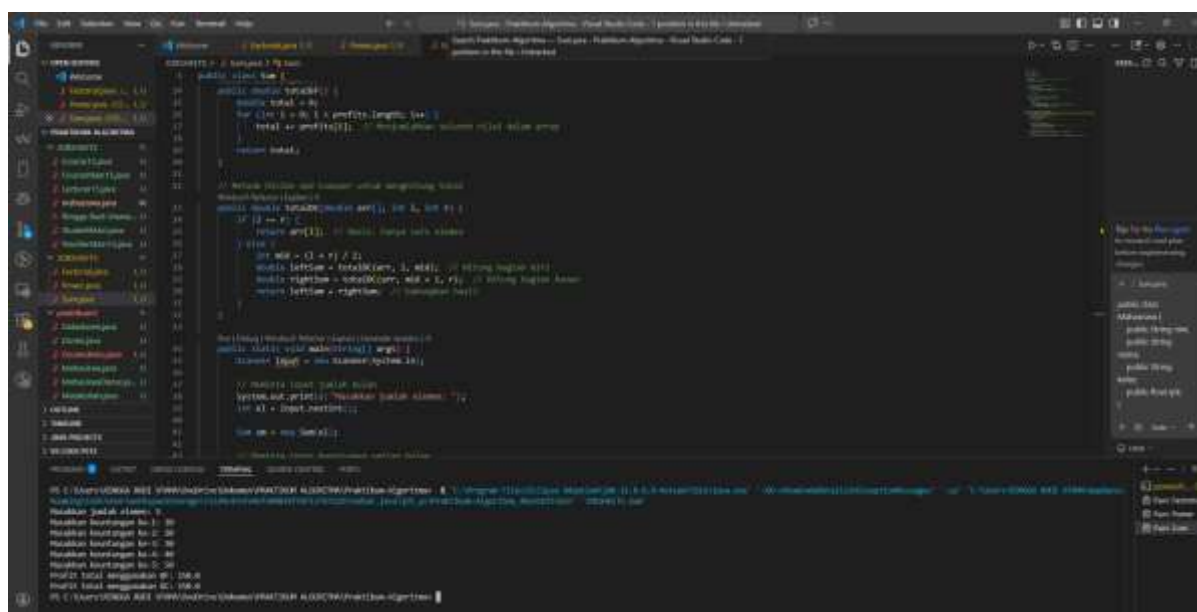
```
Scanner input = new Scanner(System.in);
System.out.print(s:"Input element number: ");
int element = input.nextInt();
```

5. Create `Sum` object and assign the profit value in each array element

```
Sum sm = new Sum(element);
for(int i=0;i<element;i++){
    System.out.print("Masukkan keuntungan ke-"+(i+1)+" : ");
    sm.profits[i] = input.nextDouble();
}
```

6. Call both `totalBF()` and `totalDC()` methods!

```
System.out.println("Profit total using BF: "+sm.totalBF());
System.out.println("Profit total using DC: "+sm.totalDC(sm.profits,l:0,element-1));
```





5.4.2. Verification of Experiment Results

Verify the results and it must be matched with the following!

```
Input element number: 5
Input profit #1: 10
Input profit #2: 20
Input profit #3: 30
Input profit #4: 40
Input profit #5: 50
Profit total using BF: 150.0
Profit total using DC: 150.0
```

```
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\PRAKTIKUM ALGORITMA\Praktikum-Algoritma> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.8.9-hot-
Roaming\Code\User\workspaceStorage\c5328c47efed4f309019f50f1379722d\redhat.java\jdk_ws\Praktikum-Algoritma_48a1bf5\bin' 'JOBSHEETS.Sum'
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Profit total menggunakan BF: 150.0
Profit total menggunakan DC: 150.0
PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\PRAKTIKUM ALGORITMA\Praktikum-Algoritma>
```

5.4.3. Questions

1. Why is `mid` variable needed in `totalDC()` method?
2. Explain the following statements in `totalDC()` method?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

3. Why is it necessary to sum the results of `lsum` and `rsum` as shown below??

```
return lsum+rsum;
```

4. What is the base case of `totalDC()` method?
5. Draw a conclusion about how `totalDC()` works!

Answer :

1. Mengapa variabel `mid` dibutuhkan dalam metode `totalDC()`?

- Variabel `mid` digunakan untuk membagi array menjadi dua bagian yang lebih kecil. Ini memungkinkan kita untuk menghitung total secara efisien dengan memecah masalah besar menjadi sub-masalah yang lebih kecil.

2. Jelaskan pernyataan berikut dalam metode `totalDC()`

- `lsum` menghitung total untuk bagian kiri array, dari indeks `l` sampai `mid`.
- `rsum` menghitung total untuk bagian kanan array, dari indeks `mid + 1` sampai `r`.
- Kedua hasil tersebut kemudian dijumlahkan untuk mendapatkan total keseluruhan.



3. Mengapa perlu menjumlahkan hasil lsum dan rsum seperti yang ditunjukkan?

- Hasil dari bagian kiri (lsum) dan bagian kanan (rsum) perlu digabungkan karena array dibagi menjadi dua bagian, dan kita perlu menggabungkan hasil dari kedua bagian untuk mendapatkan total keseluruhan.

4. Apa dasar kasus dari metode totalDC()?

- **Dasar kasus** adalah ketika hanya ada satu elemen dalam array, yaitu ketika indeks kiri (l) dan kanan (r) sama. Pada kondisi ini, nilai elemen tersebut langsung dikembalikan.

5. Buat kesimpulan tentang bagaimana totalDC() bekerja!

- **totalDC()** bekerja dengan memecah masalah menjadi dua bagian (Divide), menghitung total setiap bagian secara rekursif (Conquer), dan menggabungkan hasilnya (Combine) untuk menghasilkan total akhir.

5.5. Assignments

A university has a list of student grades with data as shown in the table below.

Name	Student ID (NIM)	Year of Admission	Midterm Score (UTS)	Final Score (UAS)
Ahmad	220101001	2022	78	82
Budi	220101002	2022	85	88
Cindy	220101003	2021	90	87
Dian	220101004	2021	76	79
Eko	220101005	2023	92	95
Fajar	220101006	2020	88	85
Gina	220101007	2023	80	83
Hadi	220101008	2020	82	84

- Find the highest Midterm Score (UTS) using the Divide and Conquer approach.
- Find the lowest Midterm Score (UTS) using the Divide and Conquer approach.
- Calculate the average Final Score (UAS) of all students using the Brute Force approach.

a) Mencari Nilai UTS Tertinggi menggunakan Divide and Conquer

Untuk mencari **nilai UTS tertinggi**, kita akan membagi array nilai **UTS** menjadi dua bagian, kemudian mencari nilai maksimum di masing-masing bagian secara rekursif dan menggabungkannya untuk mendapatkan nilai tertinggi.

Penjelasan Algoritma Divide and Conquer untuk Mencari Nilai Maksimum:

- Bagi array menjadi dua bagian.
- Hitung nilai maksimum dari masing-masing bagian secara rekursif.
- Gabungkan hasilnya untuk mendapatkan nilai maksimum global.

b) Mencari Nilai UTS Terendah menggunakan Divide and Conquer

Serupa dengan pencarian nilai tertinggi, kita akan mencari nilai **UTS terendah** dengan membagi array dan mencari nilai minimum di setiap bagian, kemudian menggabungkan hasilnya.

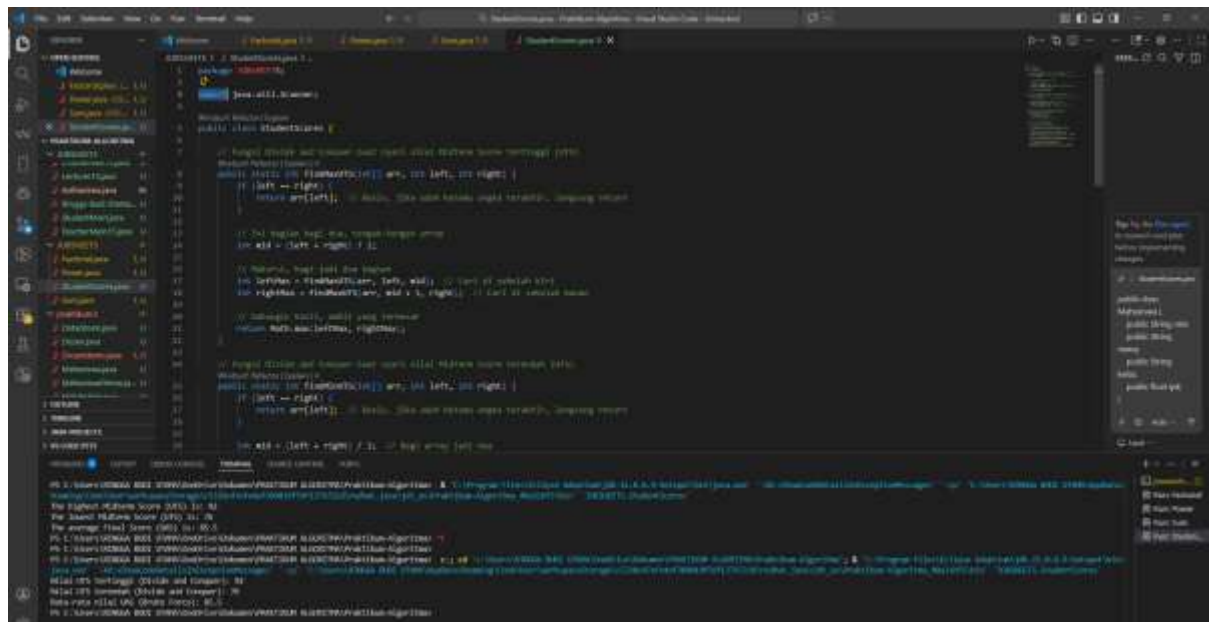
Penjelasan Algoritma Divide and Conquer untuk Mencari Nilai Minimum:

- Bagi array menjadi dua bagian.
- Hitung nilai minimum dari masing-masing bagian secara rekursif.
- Gabungkan hasilnya untuk mendapatkan nilai minimum global.

c) Menghitung Rata-rata UAS menggunakan Brute Force

Untuk menghitung rata-rata **Final Score (UAS)**, kita akan menghitung total dari semua nilai UAS dan membaginya dengan jumlah mahasiswa. Pendekatan ini menggunakan metode **Brute Force**, yaitu iterasi langsung.

Implementation in code :



```
1 // Algoritma Divide and Conquer untuk mencari nilai maksimum
2
3 // Fungsi rekursif untuk mencari nilai maksimum
4 int findMax(int arr[], int left, int right) {
5     // Base case: jika array hanya memiliki satu elemen
6     if (left == right) {
7         return arr[left];
8     }
9
10    // Bagi array menjadi dua bagian
11    int mid = (left + right) / 2;
12
13    // Cari nilai maksimum di bagian kiri
14    int leftMax = findMax(arr, left, mid);
15
16    // Cari nilai maksimum di bagian kanan
17    int rightMax = findMax(arr, mid + 1, right);
18
19    // Gabungkan hasil dari kedua bagian
20    return Math.max(leftMax, rightMax);
21 }
22
23 // Fungsi utama untuk menguji algoritma
24 public static void main(String[] args) {
25     // Array contoh
26     int arr[] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
27
28     // Mencari nilai maksimum
29     int max = findMax(arr, 0, arr.length - 1);
30
31     // Menampilkan hasil
32     System.out.println("Nilai maksimum adalah: " + max);
33 }
```